

UNIFIED MENTOR

Name	::	PRAVEEN KUMAR MOPURU
Type of Intern	::	MACHINE LEARNING INTERN
UNID	::	UMID05122573240
Batch	::	10thDec2025 till 10thMarch2026
Email	::	mpk339@gmail.com
Mobile	::	9985772150

Project 1: Predict Vehicle Prices using Vehicle Dataset

Objective: Build a System that can predict the prices for vehicles using the data on various vehicle specifications like, make, etc. Explore the data to understand the features and figure out an approach

GitHub Link to Project -1 :

[https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP1/UM_MLIntern_P1\(6Feb2026%20-3\).ipynb](https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP1/UM_MLIntern_P1(6Feb2026%20-3).ipynb)

Dataset Description: 17 features and 1002 reords

```
.. <class 'pandas.core.frame.DataFrame'>
RangeIndex: 1002 entries, 0 to 1001
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   name                  1002 non-null   object
1   description            946 non-null    object
2   make                  1002 non-null   object
3   model                 1002 non-null   object
4   year                  1002 non-null   int64
5   price                 979 non-null    float64
6   engine                1000 non-null   object
7   cylinders              897 non-null    float64
8   fuel                  995 non-null    object
9   mileage               968 non-null    float64
10  transmission           1000 non-null   object
11  trim                   1001 non-null   object
12  body                   999 non-null    object
13  doors                  995 non-null    float64
14  exterior_color         997 non-null    object
15  interior_color         964 non-null    object
16  drivetrain             1002 non-null   object
dtypes: float64(4), int64(1), object(12)
memory usage: 133.2+ KB
```

Target : Price

World cloud of Description Column:



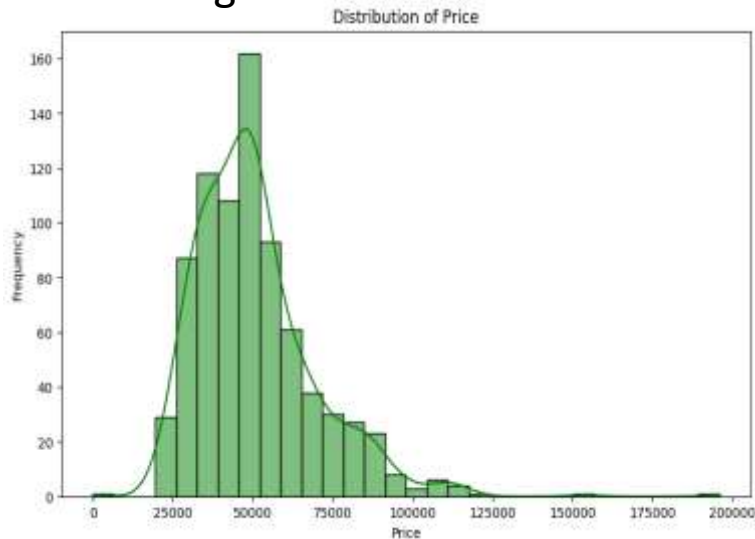
The most common repeating words in the description are :

Vehicle
Steering Wheel,
Speed,
Automatic,
Price,

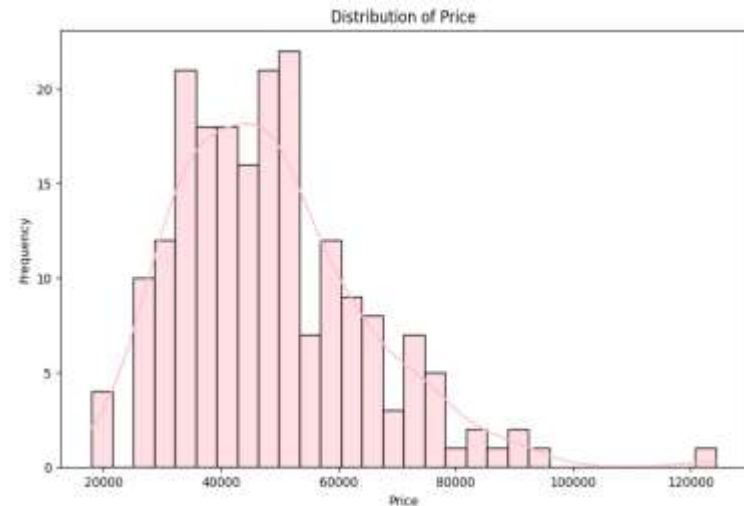
Problem Type:

Predicting the price, Continuous variable is a Regression Problem

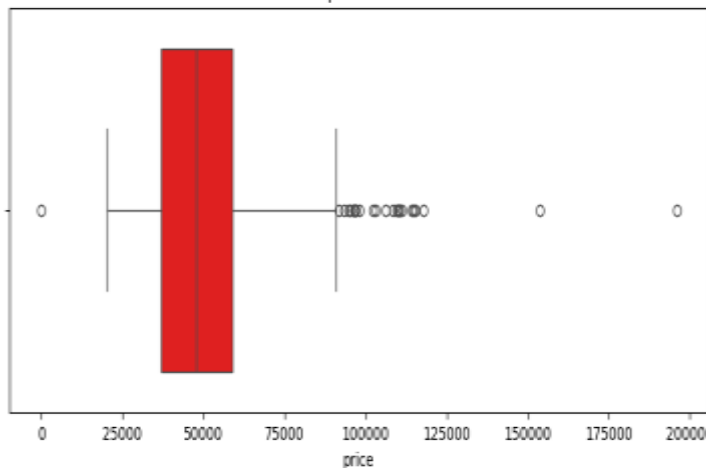
Training set Distribution - Price



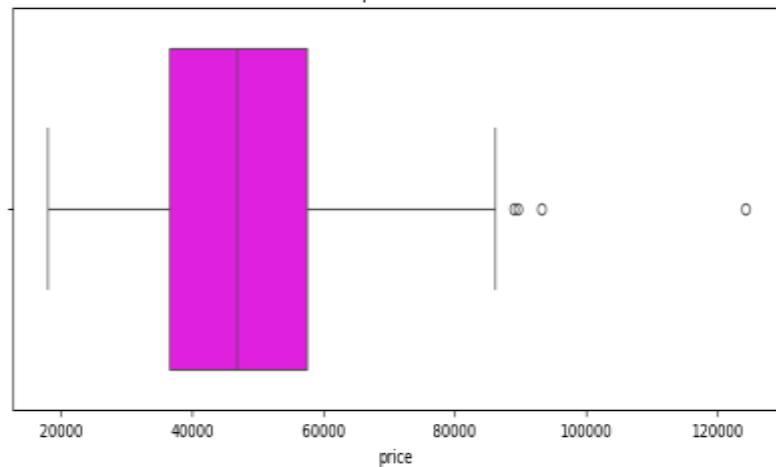
Testing set Distribution - Price



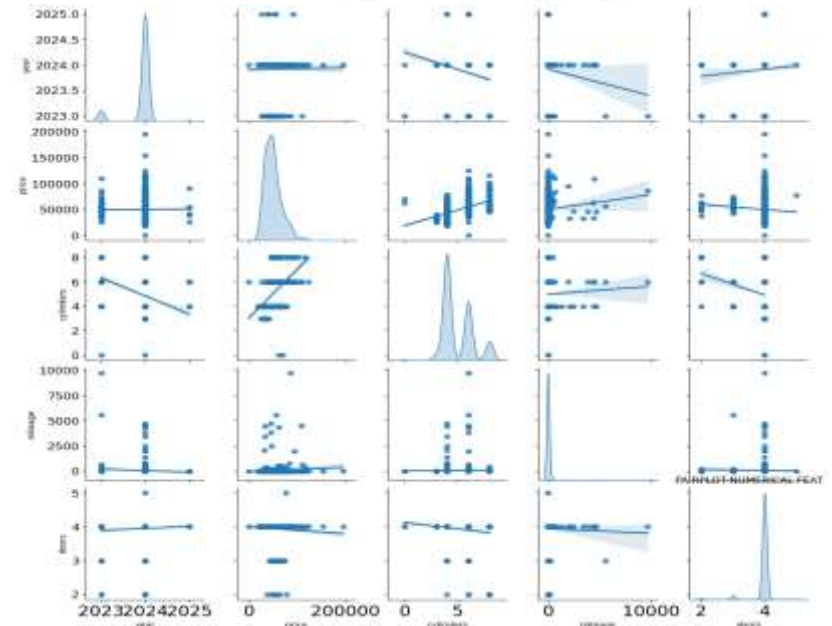
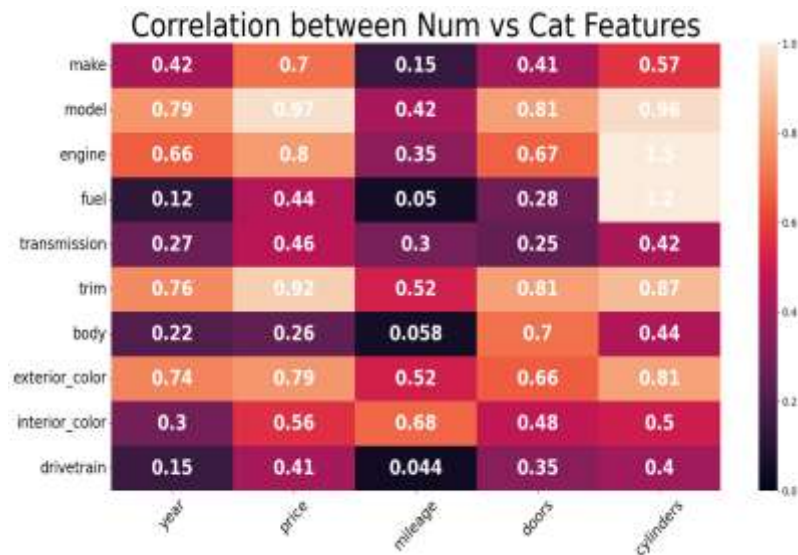
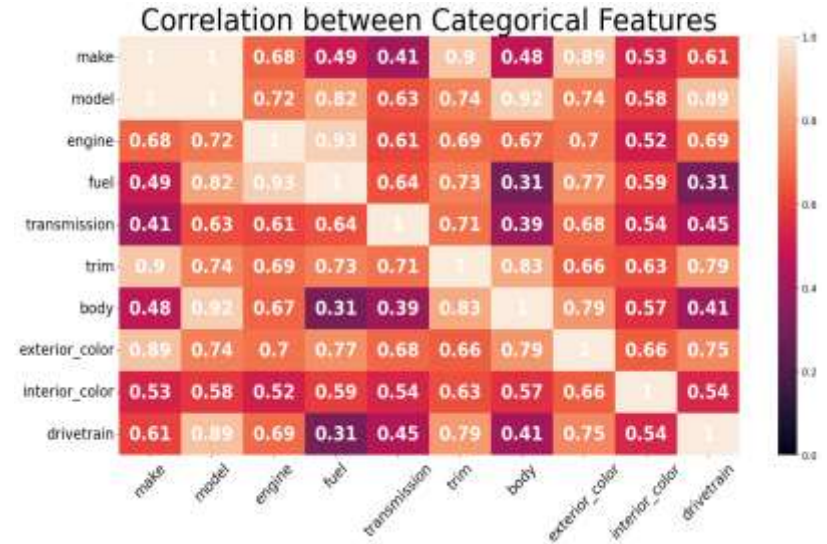
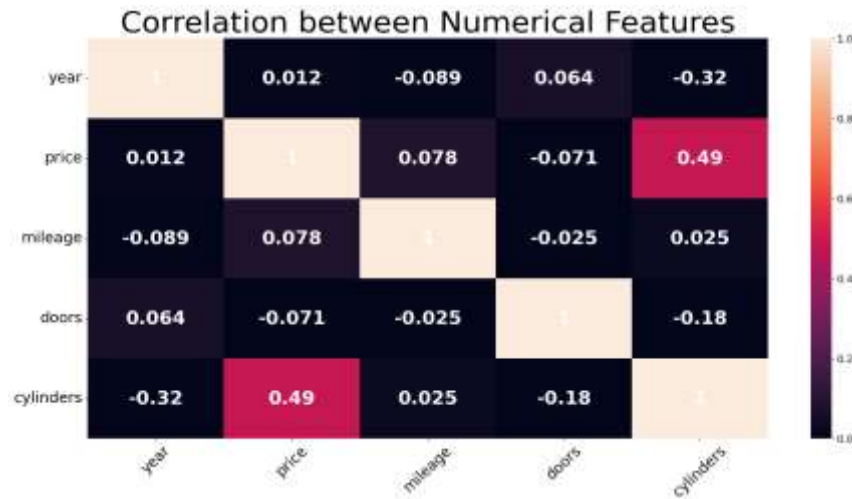
Boxplot of Price



Boxplot of Price



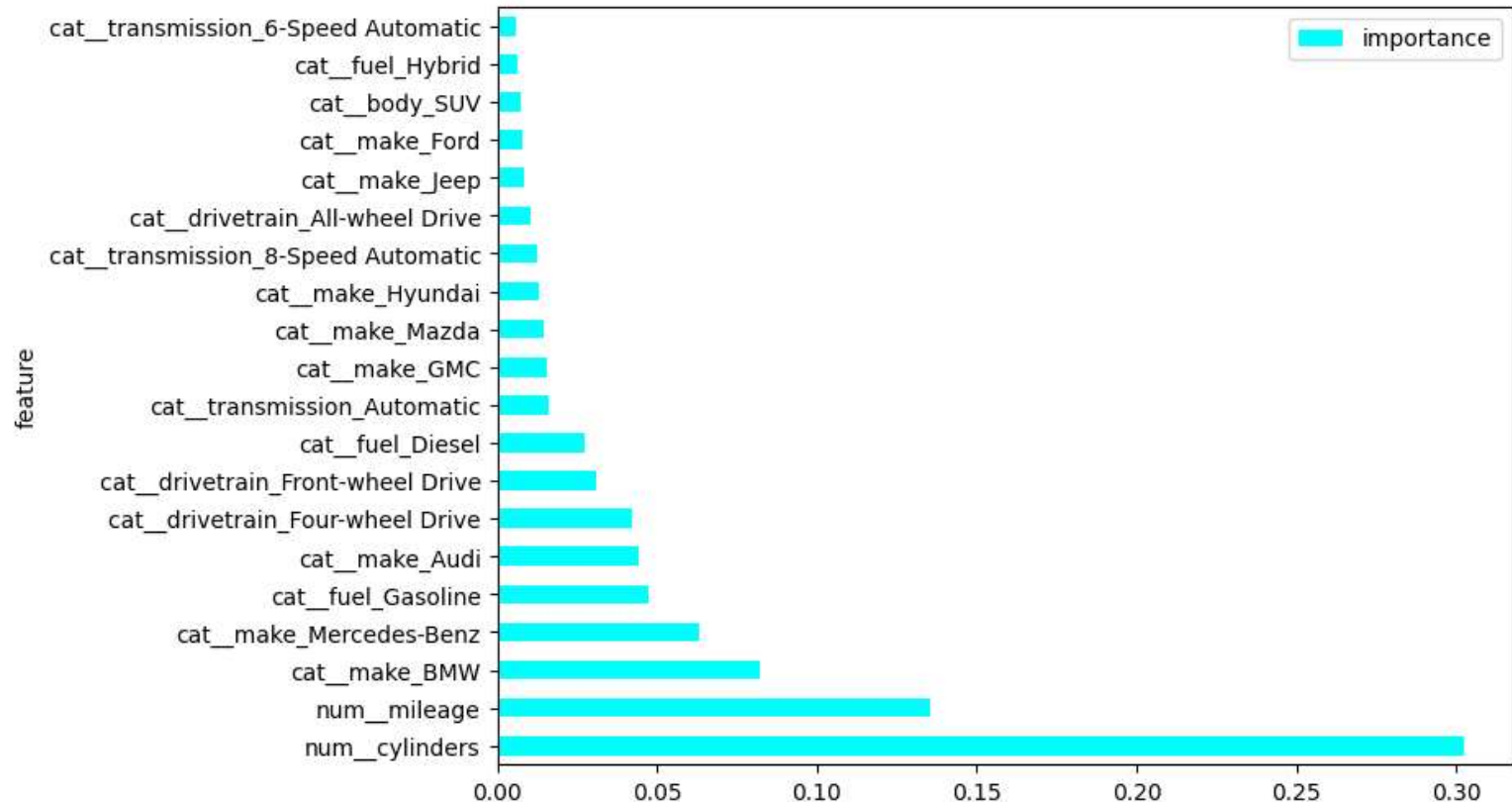
Exploratory Data Analysis & Pairplot of features



MODEL BUILDING FOR PREDICTING THE PRICE

S.No	ML Algorithm	Model Name	R ²	RMSE(USD)	Predicted Price in USD
1.	OLS from statsmodel	modelOLS1	0.4248	--	--
2.	Random Forest	modelLR	0.6423	9549.39	42279.28
3.	Linear Regression	model_lr	0.4876	11429	59291.67
4.	XGBoost	model_xgb	0.6484	9467.37	52482.83
5.	Ridge	meta_model	0.6484	9467.37	48213.73
6.	Lasso	model_lasso	0.4878	11427.59	60049.13
7.	ElasticNet	modl_ENet	0.5090	11187.95	63575.08
8.	ANN	modelANN	0.4777	11539.32	61800.53
9.	LightGBM	modelLGBM	--	--	41831.08
10.	Linear regression (All features in the Dataset)	lr_model	- 0.5456	19849.99	36772.84

Feature Importances Vehicle



From the above figure we can observe that Num of cylinders, mileage of the vehicle, Make of vehicle (BMW, Benz, Audi), Fuel Type, Drive Train, transmission are the very essential feature of the given Dataset.

The hierarchy of Feature Importance obtained by Permutation Importance

feature	importance
cylinders	0.477597
make	0.295094
fuel	0.290675
drivetrain	0.199938
mileage	0.111276
transmission	0.108395
body	0.005669
year	0.002408
doors	-0.000054

[167]:

from the above list we can observe that cyliders make fuel drivetrain are some of the top essential feature customers look for while purchasing the vehicle

Checking the Multi-Collinearity b/n features

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

vif = pd.DataFrame()
vif['feature'] = X.columns
vif['VIF'] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
print(vif.sort_values('VIF', ascending=False).head(10))
```

	feature	VIF
5	fuel_Diesel (B20 capable)	inf
15	transmission_10-Speed Shiftable Automatic	inf
0	const	6.097718e+07
29	transmission_8-Speed Automatic	2.213358e+02
42	transmission_Automatic	1.818765e+02
19	transmission_6-Speed Automatic	8.997972e+01
11	transmission_1-Speed Automatic	6.698967e+01
38	transmission_9-Speed Automatic	5.934790e+01
13	transmission_10-Speed Automatic	5.590460e+01
13	transmission_Automatic CVT	4.401094e+01

From the above figure we can see that fuel type and drivetrain are highly collinear so one of the feature can be reduced in order to increase the model performance

RESULT & CONCLUSION:

Certainly we can observe that XGBoost model definitely perform well compared to other models, with R^2 value of **0.6484** which is **65%**

Link to Project -1

[https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP1/UM_MLIntern_P1\(6Feb2026%20-3\).ipynb](https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP1/UM_MLIntern_P1(6Feb2026%20-3).ipynb)

End of Project 1

UNIFIED MENTOR

Name	::	PRAVEEN KUMAR MOPURU
Type of Intern	::	MACHINE LEARNING INTERN
UNID	::	UMID05122573240
Batch	::	10thDec2025 till 10thMarch2026
Email	::	mpk339@gmail.com
Mobile	::	9985772150

Project 2: Fraud Transaction Detection Using Transactions Dataset

Objective: Build system that can classify if a transaction is fraudulent or not

Github Link to Project -2:

[https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP2/UM_MLIntern_P2\(17Feb2026-3\).ipynb](https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP2/UM_MLIntern_P2(17Feb2026-3).ipynb)

Dataset Description:

9 Features and 1754155 Records over 183 pickle files combined.

```
1 final_dataframe.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1754155 entries, 0 to 1754154
Data columns (total 9 columns):
 #   Column                Dtype
---  -
 0   TRANSACTION_ID        int64
 1   TX_DATETIME            datetime64[ns]
 2   CUSTOMER_ID           object
 3   TERMINAL_ID           object
 4   TX_AMOUNT             float64
 5   TX_TIME_SECONDS       object
 6   TX_TIME_DAYS          object
 7   TX_FRAUD              int64
 8   TX_FRAUD_SCENARIO     int64
dtypes: datetime64[ns](1), float64(1), int64(3), object(4)
memory usage: 120.4+ MB
```

Target :

Binary Classification ----> TX_FRAUD

MultiClass Classification ----> TX_FRAUD_SCENARIO

Fraud Trasaction is definetly Class Imbalance Classification Problem which can be solved using SMOTE Technique

```
1 final_dataframe['TX_FRAUD'].value_counts()
```

	count
TX_FRAUD	
0	1739474
1	14681

dtype: int64

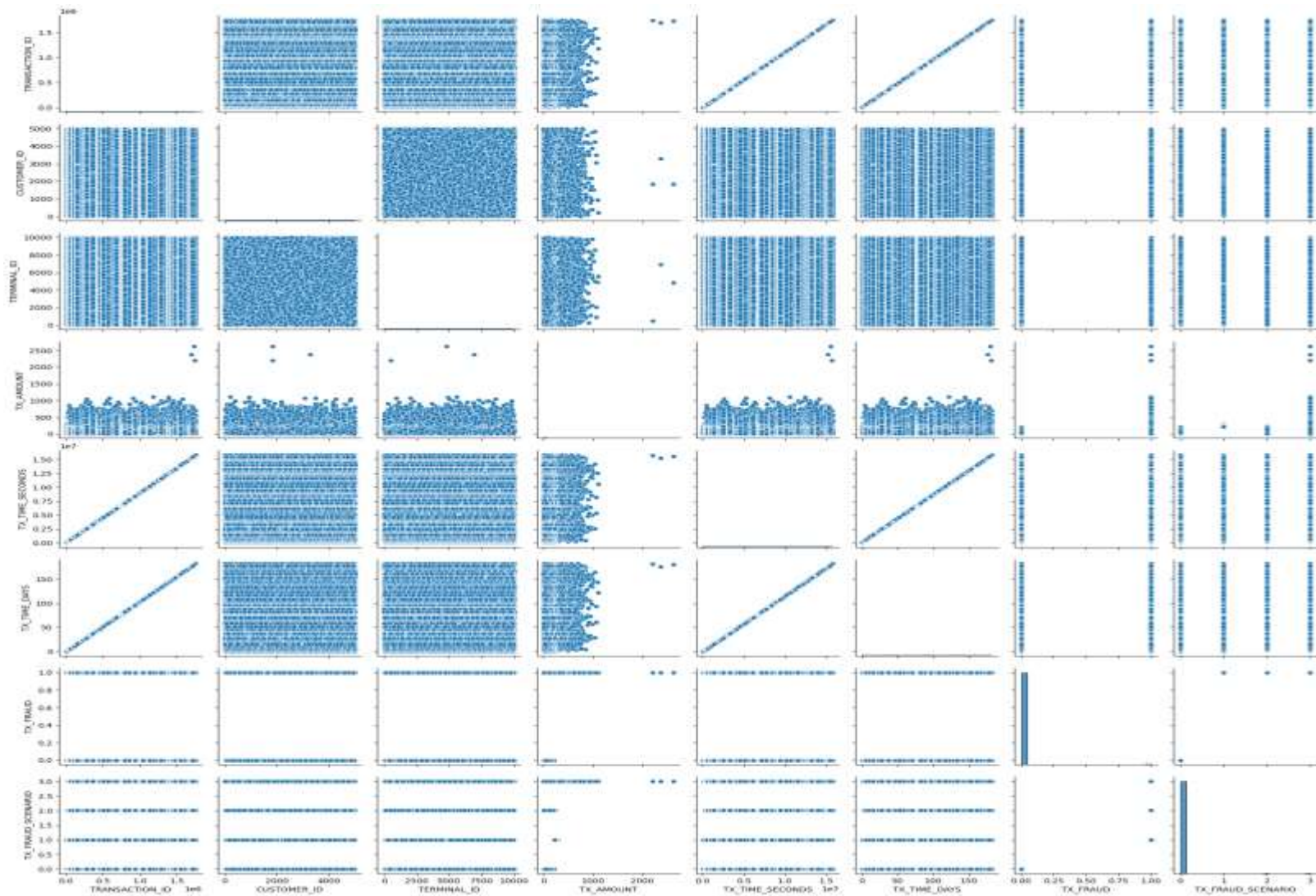
```
1 final_dataframe['TX_FRAUD_SCENARIO'].value_counts()
```

...

	count
TX_FRAUD_SCENARIO	
0	1739474
2	9077
3	4631
1	973

dtype: int64

Exploratory Data Analysis and Pairplot:



Scenario-1:

Any Transaction whose amount is more than 220 is fraud, This is not inspired by a real-world scenario. It provides an obvious fraud pattern that should be detected by any baseline fraud detector. This will be useful to validate the implementation of a fraud detection technique.

New Features Created from TX_DATETIME are **hour**, **weekday** and **day**.

```
2  final_dataframe1 = final_dataframe.copy(deep=True)
3
4  final_dataframe1["TX_DATETIME"] = pd.to_datetime(final_dataframe1["TX_DATETIME"])
5  final_dataframe1["hour"] = final_dataframe1["TX_DATETIME"].dt.hour
6  final_dataframe1["weekday"] = final_dataframe1["TX_DATETIME"].dt.weekday
7  final_dataframe1["day"] = final_dataframe1["TX_DATETIME"].dt.day
8
9  final_dataframe1 = final_dataframe1.drop(columns=["TX_DATETIME"])
```

```
1  final_dataframe1.shape
```

```
(1754155, 11)
```

Final Dataframe size is 1754155 records with 11 features

Scenario-2:

Every Day, A list of two terminals is drawn at random. All Transactions on these terminals in the next 28 days will be marked as fraudulent. This scenario simulates a criminal use of a terminal, through phishing for example. You could add features that keep track of the number of fraudulent transactions on the terminal to help with this scenario.

```
final_df2['TERM_FRAUD_COUNT_1D'] = (  
    final_df2.groupby('TERMINAL_ID')['TX_FRAUD']  
    .rolling('1D')#, on=final_df2['TX_DATETIME']  
    .sum()  
    .reset_index(level=0, drop=True)  
)  
  
final_df2['TERM_FRAUD_COUNT_7D'] = (  
    final_df2.groupby('TERMINAL_ID')['TX_FRAUD']  
    .rolling('7D')  
    .sum()  
    .reset_index(level=0, drop=True)  
)  
  
final_df2['TERM_TX_COUNT_7D'] = (  
    final_df2.groupby('TERMINAL_ID')['TX_AMOUNT']  
    .rolling('7D')  
    .count()  
    .reset_index(level=0, drop=True)  
)  
  
final_df2['TERM_FRAUD_RATIO_7D'] = (  
    final_df2['TERM_FRAUD_COUNT_7D'] / final_df2['TERM_TX_COUNT_7D']  
).fillna(0)
```

New Feature created are :

TERM_FRAUD_COUNT_1D,
TERM_FRAUD_COUNT_7D,
TERM_TX_COUNT_7D,
TERM_FRAUD_RATIO_7D,

Scenario-3:

Every Day a list of 3 customers is drawn at random. In the next 14 Days, 1/3 of their transactions have their amounts multiplied by 5 and marked as fraudulent. This scenario simulates a card-not-present fraud where the Credentials of a customer have been leaked. The customer continues to make transactions, and transactions of higher values are made by the fraudsters who tries to maximize gains,

You could add features that keep track of the spending habits of the

```
1 # Calculating Rolling Average Amount
2
3 final_df3['CUST_AVG_AMOUNT'] = final_df3.groupby('CUSTOMER_ID')['TX_AMOUNT'].transform(lambda x:x.expanding().mean())
```

```
1 final_df3['TX_Z_SCORE'] = (final_df3['TX_AMOUNT'] - final_df3['mean'])/(final_df3['std'])
2 final_df3['TX_Z_SCORE'].head(5)
```

Scenario-3:

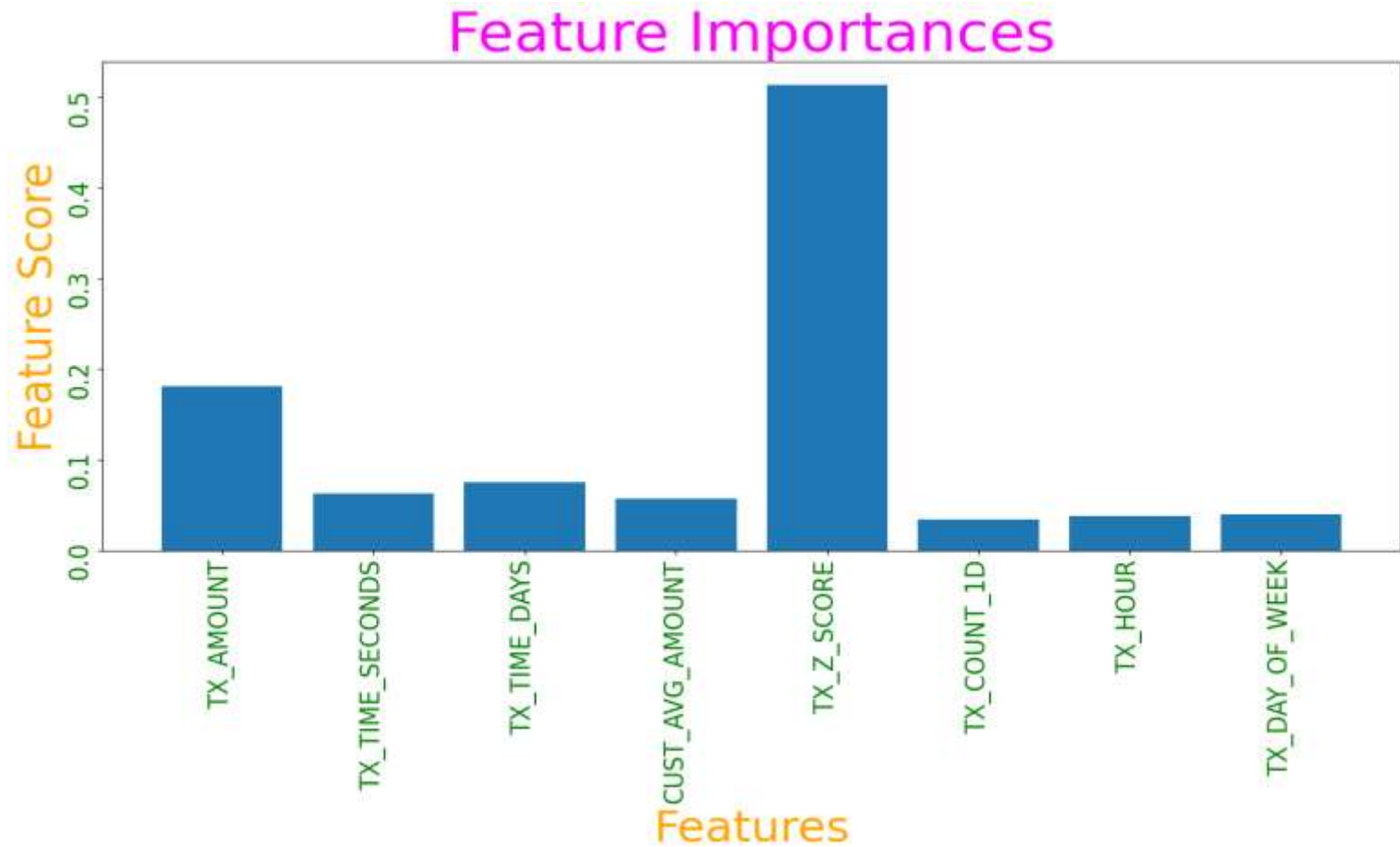
New Features created are

CUST_AVG_AMOUNT, TX_Z_SCORE,
TX_COUNT_1D, TX_DAY_OF_WEEK

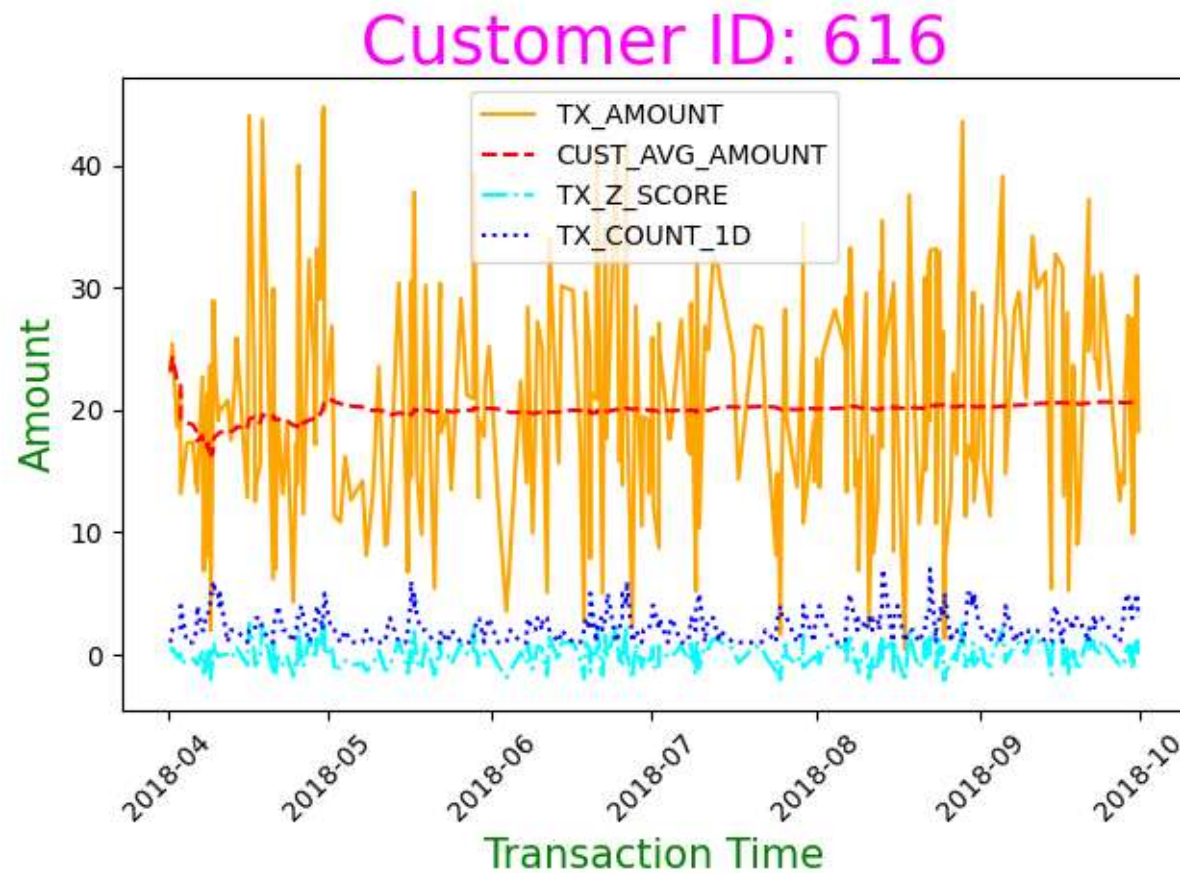
```
> 1 final_df3.info()

• <class 'pandas.core.frame.DataFrame'>
  RangeIndex: 1754155 entries, 0 to 1754154
  Data columns (total 15 columns):
   #   Column                Dtype
  ---  -
   0   TRANSACTION_ID        int64
   1   TX_DATETIME            datetime64[ns]
   2   CUSTOMER_ID           int64
   3   TERMINAL_ID           int64
   4   TX_AMOUNT             float64
   5   TX_TIME_SECONDS       int64
   6   TX_TIME_DAYS          int64
   7   TX_FRAUD              int64
   8   TX_FRAUD_SCENARIO     int64
   9   CUST_AVG_AMOUNT       float64
  10   mean                  float64
  11   std                   float64
  12   TX_Z_SCORE            float64
  13   TX_COUNT_2D           int64
  14   TX_COUNT_1D           int64
  dtypes: datetime64[ns](1), float64(5), int64(9)
  memory usage: 200.7 MB
```

Feature importances from XGBoost Model



Plotting Customer-ID of Fraud/Legitimate Transaction



RESULT & CONCLUSION:

Definetly the fraud trasactions of all the 3 scenarios combined together is 14681 out of 1754155 samples/records done by the customers, which is 1% of the Total Trasactions done by the customer but the amount of Transactions is higher.

Because the Fraud Classes are very less compared to the Legimate Trasactions, there is class imbalance which is resolved using the SMOTE trchnique, other wise the model may get biased.

Link to Project-2

[https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP2/UM_MLIntern_P2\(17Feb2026-3\).ipynb](https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP2/UM_MLIntern_P2(17Feb2026-3).ipynb)

End of Project-2

UNIFIED MENTOR

Name	::	PRAVEEN KUMAR MOPURU
Type of Intern	::	MACHINE LEARNING INTERN
UNID	::	UMID05122573240
Batch	::	10thDec2025 till 10thMarch2026
Email	::	mpk339@gmail.com
Mobile	::	9985772150

Project 3: Predict Mobile Phone Pricing

Objective: Build system that can predict pricing for a mobile phone using data on available phones in the market. Predict if the mobile can be priced Low/Med/High/VeryHigh. Explore the data to understand the features and figure out an approach.

Github Link to Project -3 :

[https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP3/UM_MLIntern_P3\(8Feb2026-4\).ipynb](https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP3/UM_MLIntern_P3(8Feb2026-4).ipynb)

Dataset Description: Tha dataset contains 2000 records 21 Features

```
1 DfMPP.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   battery_power         2000 non-null   int64
 1   blue                   2000 non-null   int64
 2   clock_speed           2000 non-null   float64
 3   dual_sim              2000 non-null   int64
 4   fc                     2000 non-null   int64
 5   four_g                2000 non-null   int64
 6   int_memory            2000 non-null   int64
 7   m_dep                 2000 non-null   float64
 8   mobile_wt             2000 non-null   int64
 9   n_cores               2000 non-null   int64
10   pc                     2000 non-null   int64
11   px_height             2000 non-null   int64
12   px_width              2000 non-null   int64
13   ram                   2000 non-null   int64
14   sc_h                  2000 non-null   int64
15   sc_w                  2000 non-null   int64
16   talk_time             2000 non-null   int64
17   three_g               2000 non-null   int64
18   touch_screen          2000 non-null   int64
19   wifi                  2000 non-null   int64
20   price_range           2000 non-null   int64
dtypes: float64(2), int64(19)
memory usage: 328.3 KB
```

Target: Price_range

Price range is a
multi class classfication
problem type

MODEL -1 :: LOGISTIC REGRESSION

```
1 from sklearn.linear_model import LogisticRegression
```

```
1 LR = LogisticRegression(max_iter = 500,  
2 | | | | | | | | | | multi_class = "multinomial")  
3  
4 LR.fit(X_train_Scaled, y_train)
```

```
LogisticRegression  
LogisticRegression(max_iter=500, multi_class='multinomial')
```

LR Train Accuracy : 0.976875
Precision: 0.9769172694622088
Recall : 0.976875
F1 Score : 0.976791202687239
ROC AUC : 1.0

(LR) CONFUSION MATRIX - TRAIN

Confusion Matrix:

```
[[398  2  0  0]  
 [ 10 385  5  0]  
 [  0  8 382 10]  
 [  0  0  2 398]]
```

(LR) CLASSIFICATION REPORT - TRAIN

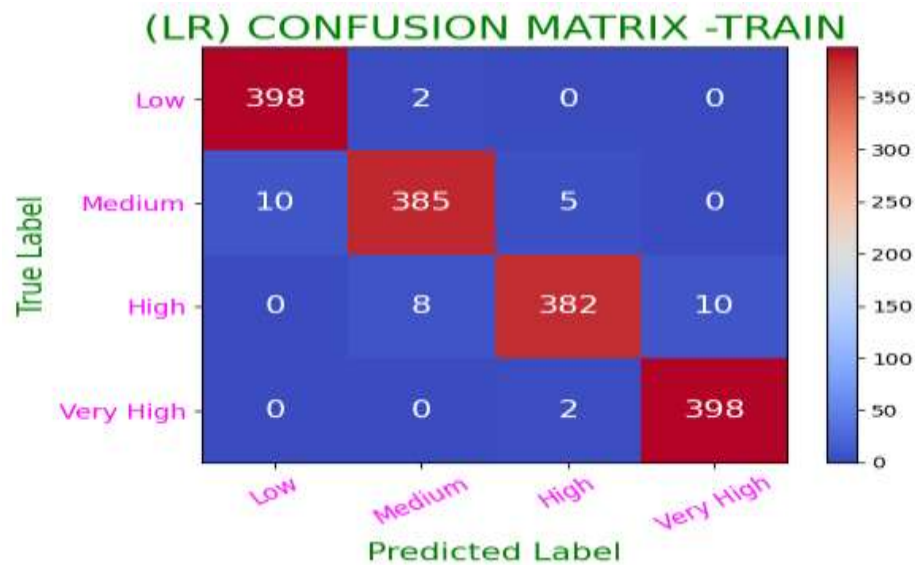
	precision	recall	f1-score	support
Low	0.98	0.99	0.99	400
Medium	0.97	0.96	0.97	400
High	0.98	0.95	0.97	400
Very High	0.98	0.99	0.99	400
accuracy			0.98	1600
macro avg	0.98	0.98	0.98	1600
weighted avg	0.98	0.98	0.98	1600

MODEL -1 :: LOGISTIC REGRESSION

```
1 from sklearn.linear_model import LogisticRegression
```

```
1 LR = LogisticRegression(max_iter = 500,  
2 | | | | | | | | | | multi_class = "multinomial")  
3  
4 LR.fit(X_train_Scaled, y_train)
```

```
LogisticRegression  
LogisticRegression(max_iter=500, multi_class='multinomial')
```



MODEL -1 :: LOGISTIC REGRESSION

```
1 from sklearn.linear_model import LogisticRegression
```

```
1 LR = LogisticRegression(max_iter = 500,  
2 | | | | | | | | | | multi_class = "multinomial")  
3  
4 LR.fit(X_train_Scaled, y_train)
```

```
LogisticRegression  
LogisticRegression(max_iter=500, multi_class='multinomial')
```

LR Test Accuracy : 0.965
Precision: 0.9650445632798573
Recall : 0.965
F1 Score : 0.9649863177272501
ROC AUC : 0.9986666666666667

(LR) CONFUSION MATRIX - TEST

Confusion Matrix:

```
[[98  2  0  0]  
 [ 1 96  3  0]  
 [ 0  2 94  4]  
 [ 0  0  2 98]]
```

(LR) CLASSIFICATION REPORT - TEST

	precision	recall	f1-score	support
Low	0.99	0.98	0.98	100
Medium	0.96	0.96	0.96	100
High	0.95	0.94	0.94	100
Very High	0.96	0.98	0.97	100
accuracy			0.96	400
macro avg	0.97	0.96	0.96	400
weighted avg	0.97	0.96	0.96	400

MODEL -1 :: LOGISTIC REGRESSION

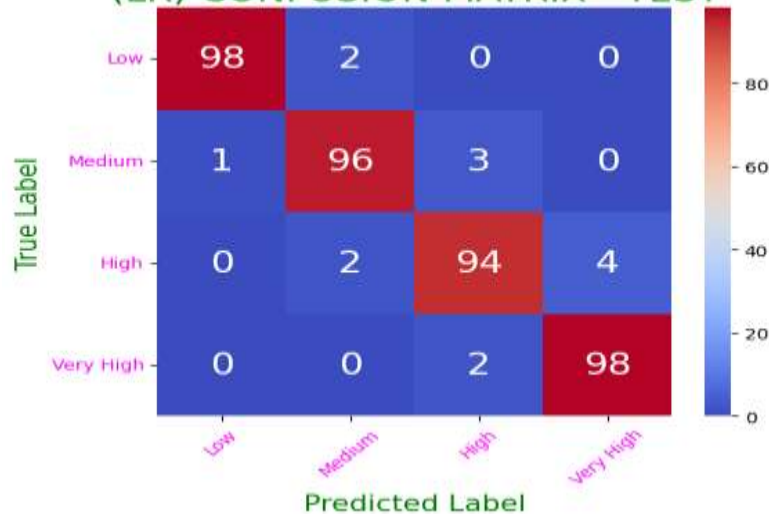
```
1 from sklearn.linear_model import LogisticRegression
```

```
1 LR = LogisticRegression(max_iter = 500,  
2 | | | | | | | | | | multi_class = "multinomial")  
3  
4 LR.fit(X_train_Scaled, y_train)
```

LogisticRegression

```
LogisticRegression(max_iter=500, multi_class='multinomial')
```

(LR) CONFUSION MATRIX - TEST



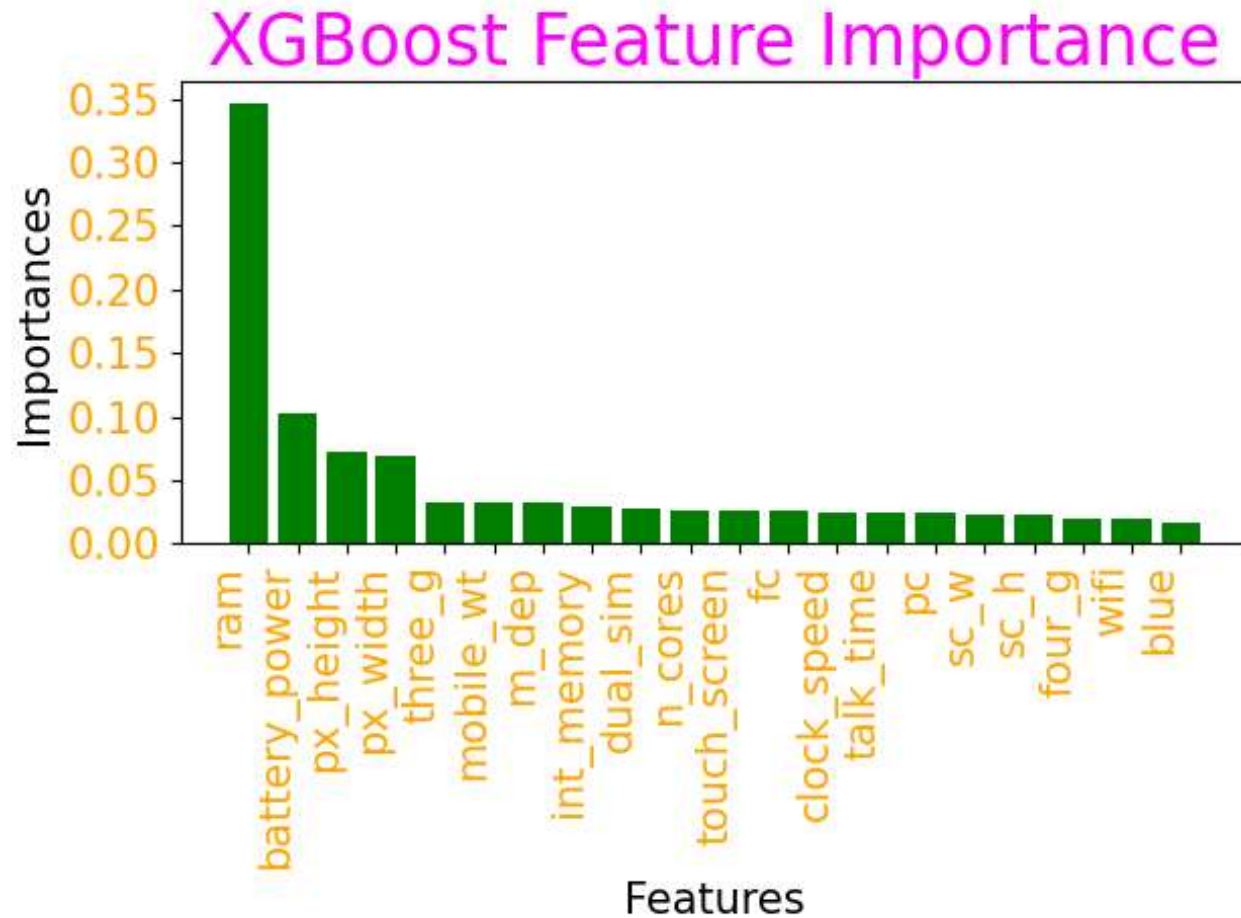
Predicting Inference for a New Mobile

```
1 New_Phone=[[1800, 1, 2.4, 1, 13, 1, 64, 0.5, 170, 8, 16, 1000, 1900, 3000, 15, 10, 14, 1, 1, 1]]  
2 NP_Scaled=scaler.transform(New_Phone)  
3 NMP = LR.predict(NP_Scaled)  
4 print(NMP)  
5  
6 if NMP==0:  
7     print("Price is Low")  
8 elif NMP==1:  
9     print("Price is Medium")  
10 elif NMP==2:  
11     print("Price is High")  
12 else:  
13     print("Price is Very High")  
  
*** [3]  
Price is Very High
```

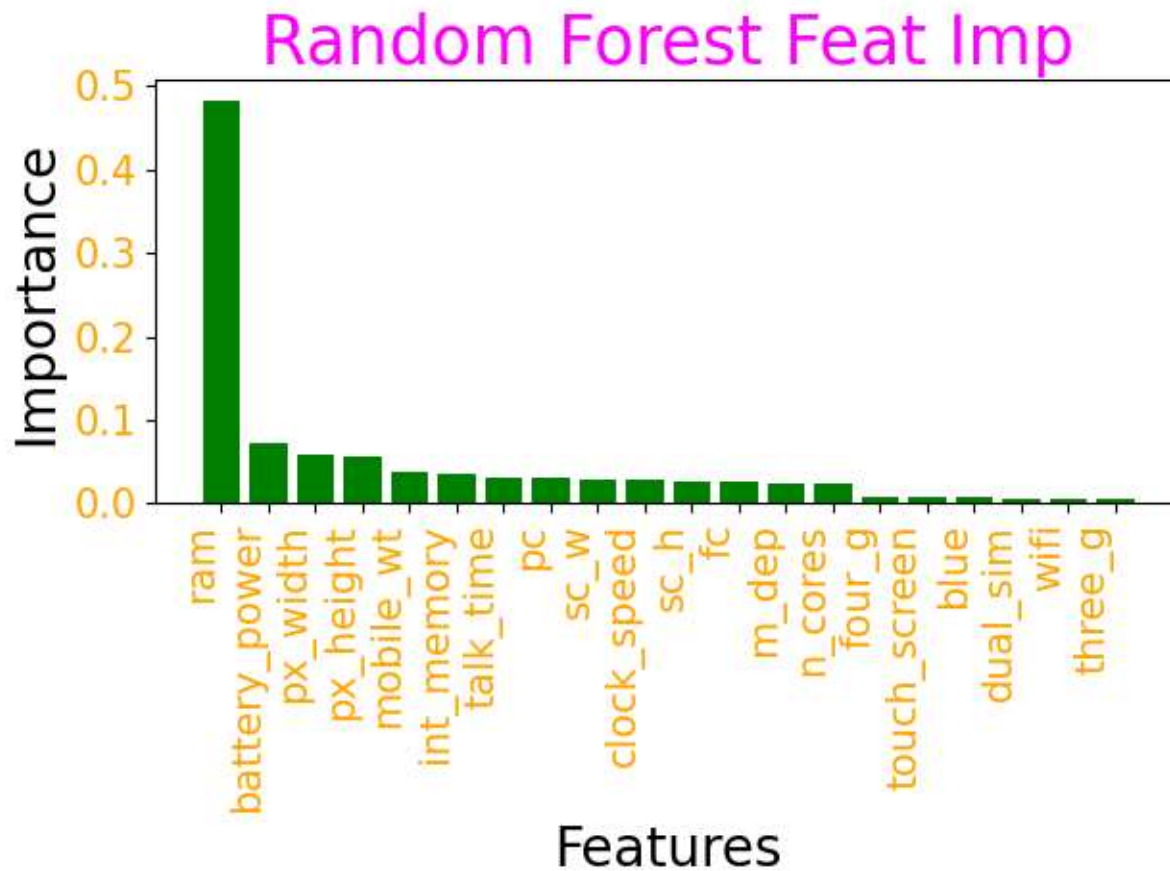
The Dataset is Perfectly Balance with 500 samples each for all the 4 types of Price Range (Low, Medium, High, Very High) with the Total of 2000 sample for each Type.



Feature Importances of the dataset from the model Logistic Regression, Random Forest & XGBoost are as Follows

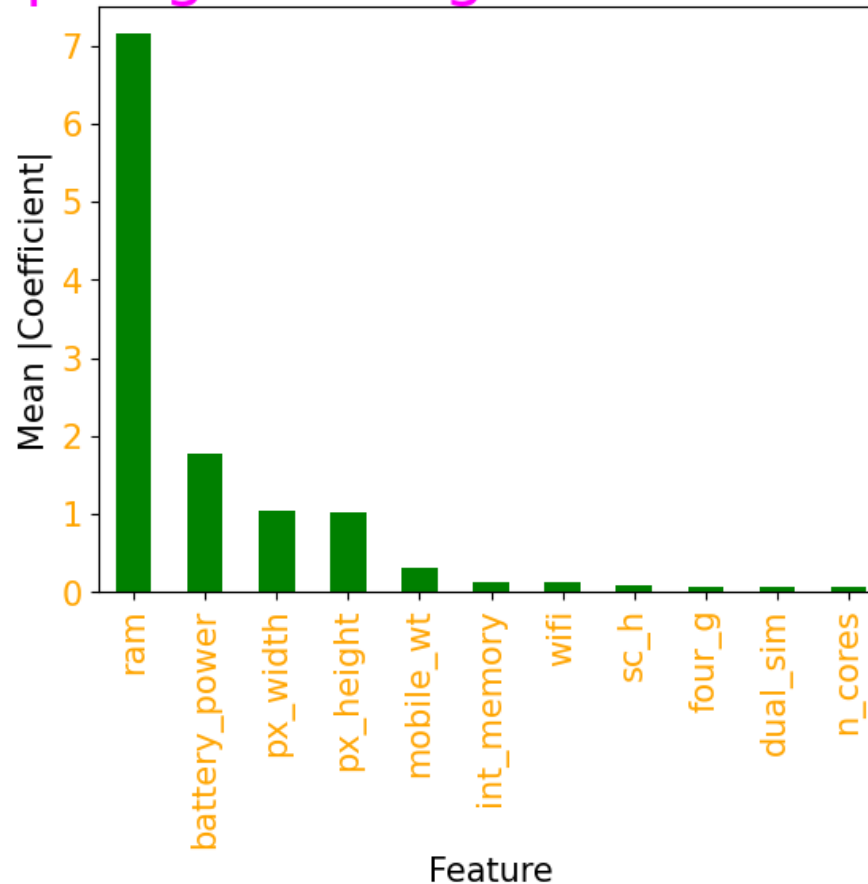


Feature Importances of the dataset from the model Logistic Regression, Random Forest & XGBoost are as Follows



Feature Importances of the dataset from the model Logistic Regression, Random Forest & XGBoost are as Follows

Top Logistic Regression Features



RESULT & CONCLUSION:

Certainly We can observe from the above three Feature Importance charts that The most Essential or the Top features which are considered by the customer while buying the mobile are found to be:

- >RAM
- >BATTERY POWER
- >PIXEL HEIGHT
- >PIXEL WIDTH
- >MOBILE WEIGHT etc.

Link to Project-3

[https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP3/UM_MLIntern_P3\(8Feb2026-4\).ipynb](https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP3/UM_MLIntern_P3(8Feb2026-4).ipynb)

End of Project -3

UNIFIED MENTOR

Name	::	PRAVEEN KUMAR MOPURU
Type of Intern	::	MACHINE LEARNING INTERN
UNID	::	UMID05122573240
Batch	::	10thDec2025 till 10thMarch2026
Email	::	mpk339@gmail.com
Mobile	::	9985772150

Project 4 : Image Classification of Animals

Objective: Build a System that can identify the animal in a given image. Explore the data set and identify an appropriate solution. Tip: You could also experiment with Neural Networks and Transfer Learning for a better learning.

GitHub Link for Project-4 :

[https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP4/UM_MLIntern_P4\(9Feb2026-6\).ipynb](https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP4/UM_MLIntern_P4(9Feb2026-6).ipynb)

Dataset Description: Dataset contains 15 classes of animals

S.No	Animal	Num of Samples
1.	Panda	135
2.	Elephant	133
3.	Lion	131
4.	Zebra	137
5.	Deer	127
6.	Horse	130
7.	Bear	125
8.	Bird	137
9.	Tiger	129
10.	Dolphin	129
11.	Cow	131
12.	Cat	123
13.	Giraffe	129
14.	Kangaroo	126
15.	Dog	122

Sample images from the Dataset::

Dog



Bear



Elephant



Horse



Deer



Giraffe



Cat



Panda



Dolphin



Cow



Bird



Lion



Kangaroo



Zebra



Tiger



Counting Number of Files In each folder

Model Building of Convolutional Neural Network:

Importing the Libraries for building CNN:

```
1  import tensorflow as tf
```

```
1
2  from tensorflow.keras import layers
3  from tensorflow.keras.layers import Dense
4  from tensorflow.keras.layers import Flatten
5  from tensorflow.keras.layers import Dropout
6  from tensorflow.keras.layers import Input
7  from tensorflow.keras import models
8  from tensorflow.keras.models import Model
9  from tensorflow.keras.layers import AveragePooling2D
10
11 from tensorflow.keras.optimizers import Adam
12
13 from tensorflow.keras.preprocessing import image_dataset_from_directory
14
15 from tensorflow.keras.preprocessing import image
16 from tensorflow.keras.preprocessing.image import load_img
17 from tensorflow.keras.preprocessing.image import img_to_array
18
19 #from sklearn.model_selection import train_test_split
20 from sklearn.metrics import classification_report
21 from sklearn.metrics import confusion_matrix
```

Model Building of Convolutional Neural Network:

```
model = models.Sequential([
    layers.Input(shape=(224,224,3)),
    data_augmentation,
    layers.Rescaling(1./255),

    layers.Conv2D(32, 3, padding="same"),# activation='relu'),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D(),

    layers.Conv2D(64, 3, padding="same"),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D(),

    layers.Conv2D(128, 3, padding="same"),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D(),

    layers.Conv2D(256, 3, padding="same"),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.MaxPooling2D(),

    layers.Flatten(),
    layers.Dense(256),
    layers.BatchNormalization(),
    layers.Activation("relu"),
    layers.Dropout(0.5),

    layers.Dense(num_classes, activation="softmax")
])
```

```
*** Model: "sequential_4"
```

Layer (type)	Output Shape	Param #
sequential_2 (Sequential)	(None, 224, 224, 3)	0
rescaling_2 (Rescaling)	(None, 224, 224, 3)	0
conv2d_8 (Conv2D)	(None, 224, 224, 32)	896
batch_normalization_13 (BatchNormalization)	(None, 224, 224, 32)	128
activation_10 (Activation)	(None, 224, 224, 32)	0
max_pooling2d_8 (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_9 (Conv2D)	(None, 112, 112, 64)	15,408
batch_normalization_14 (BatchNormalization)	(None, 112, 112, 64)	256
activation_11 (Activation)	(None, 112, 112, 64)	0
max_pooling2d_9 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_10 (Conv2D)	(None, 56, 56, 128)	73,856
batch_normalization_15 (BatchNormalization)	(None, 56, 56, 128)	512
activation_12 (Activation)	(None, 56, 56, 128)	0
max_pooling2d_10 (MaxPooling2D)	(None, 28, 28, 128)	0
conv2d_11 (Conv2D)	(None, 28, 28, 256)	295,168
batch_normalization_16 (BatchNormalization)	(None, 28, 28, 256)	1,024
activation_13 (Activation)	(None, 28, 28, 256)	0
max_pooling2d_11 (MaxPooling2D)	(None, 14, 14, 256)	0
flatten_2 (Flatten)	(None, 50176)	0
dense_13 (Dense)	(None, 256)	11,845,312
batch_normalization_17 (BatchNormalization)	(None, 256)	1,024
activation_14 (Activation)	(None, 256)	0
dropout_8 (Dropout)	(None, 256)	0
dense_14 (Dense)	(None, 15)	3,855

Total params: 13,240,527 (50.51 MB)

Trainable params: 13,239,055 (50.50 MB)

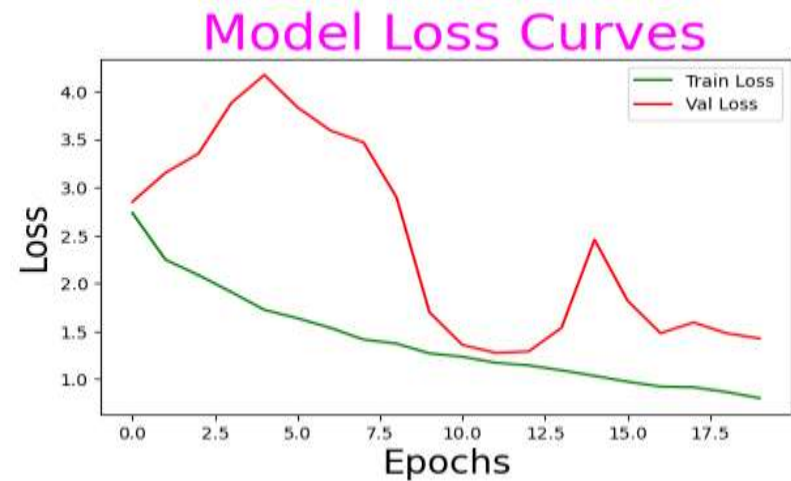
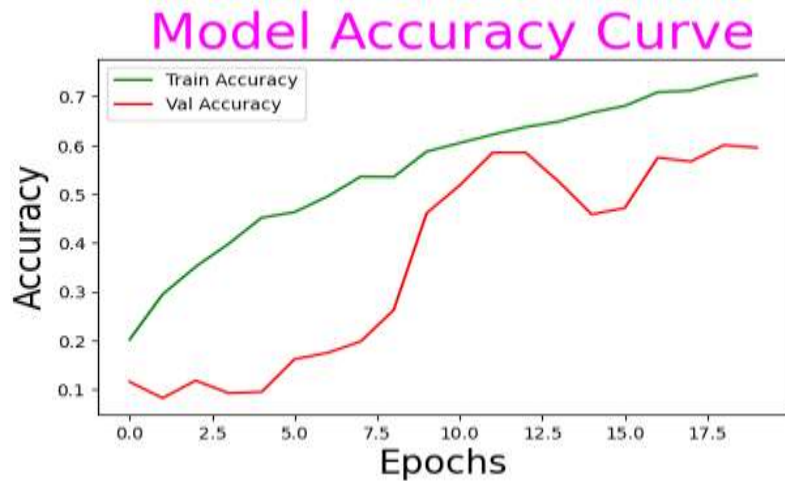
Non-trainable params: 1,472 (5.75 KB)

Total params:
13,240,527 (50.51 MB)

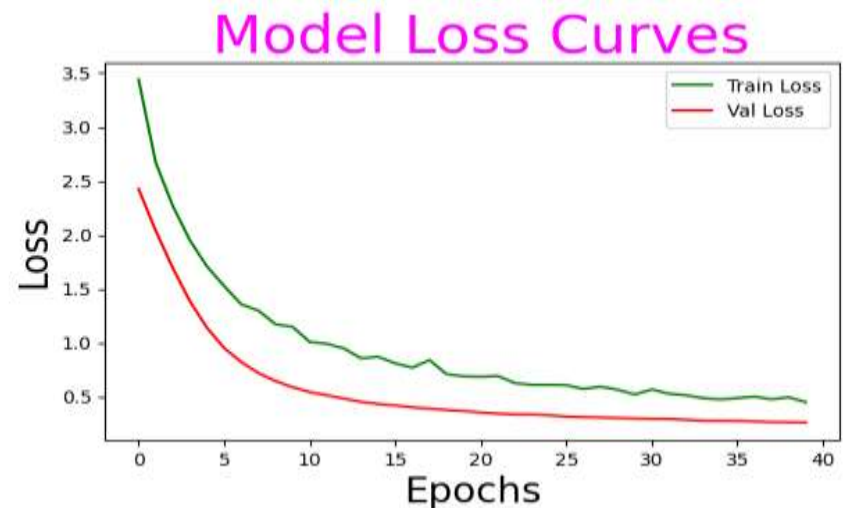
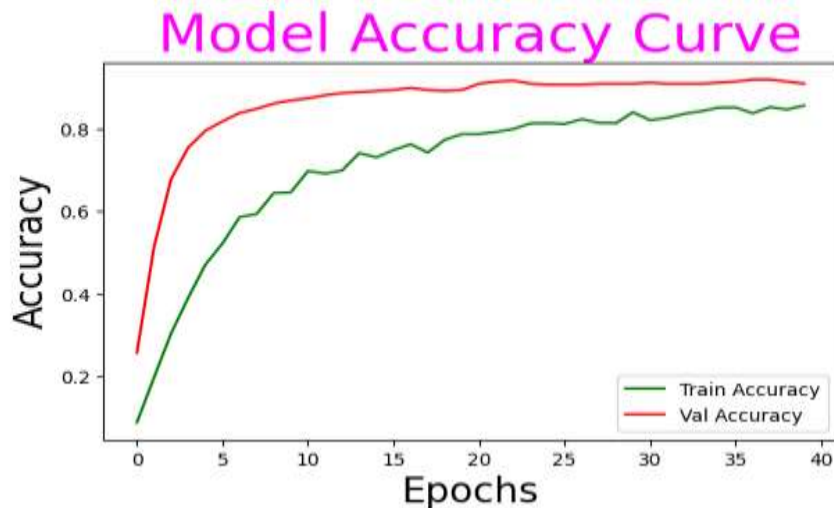
Trainable params:
13,239,055 (50.50 MB)

Non-trainable
params:
1,472 (5.75 KB)

Training the Model for 20 Epochs



Training the Model for 40 Epochs (Use GPU for Faster Training Speed and Reduced Time)



TRANSFER LEARNING

building a already pretrained VGG model with Imagenet weights & predicting the images from the Dataset

Simply Using VGG Transfer Learning Model for prediction

```
1
2 import tensorflow as tf
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from tensorflow.keras.preprocessing import image
6 from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input, decode_predictions
```

```
1 model = VGG16(weights="imagenet")
```

```
1
2 img_path = img_15# " " # change to your image
3
4 img = image.load_img(img_path, target_size=(224,224))
5 img_array = image.img_to_array(img)
6 img_array = np.expand_dims(img_array, axis=0)
7 img_array = preprocess_input(img_array)
8
9 preds = model.predict(img_array)
10 decoded = decode_predictions(preds, top=5)[0]
11
12 print("Top Predictions:")
13 for i, (imagenet_id, label, prob) in enumerate(decoded):
14     print(f"{i+1}. {label} ({prob:.4f})")
15
16 top_label = decoded[0][1]
17 top_prob = decoded[0][2]
18
19 plt.figure(figsize=(6,6))
20 plt.imshow(img)
21 plt.axis("off")
22 plt.title(f"Prediction: {top_label} ({top_prob*100:.2f}%)")
23 #plt.title(f"Prediction:", imagenet_id)#{class_names[np.argmax(decoded)]}")
24
```


Predicting the Image using Transfer Learning

```
14     print(f"{i+1}. {label} ({prob:.4f})")
15
16     top_label = decoded[0][1]
17     top_prob = decoded[0][2]
18
19     plt.figure(figsize=(6,6))
20     plt.imshow(img)
21     plt.axis("off")
22     plt.title(f"Prediction: {top_label} ({top_prob*100:.2f}%)")
23     #plt.title(f"Prediction:", imagenet_id#{class_names[np.argmax(decoded)]}")
24     plt.show()
```

```
... 1/1 ----- 0s 51ms/step
Top Predictions:
1. zebra (0.9999)
2. gazelle (0.0000)
3. hartebeest (0.0000)
4. impala (0.0000)
5. ostrich (0.0000)
```

Prediction: zebra (99.99%)



RESULT & CONCLUSION:

The model could perform better if the number of samples for each image type are more, the small number of samples for each image, makes the model a bit lagging in training for identification.

Link to Project-4:

[https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP4/UM_MLIntern_P4\(9Feb2026-6\).ipynb](https://github.com/M339KUMAR/ColabDemoFiles/blob/32ac1e1eb5380b1ad7278b95954677ff1d4d7936/UMMLP4/UM_MLIntern_P4(9Feb2026-6).ipynb)

End of Project 4